

DSA4212 Optimization Notes & CheatSheet

2025-2026-1 @Jinhang

Linear Algebra

Definition Let $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$

• Norm:

- $\|\mathbf{a}\| = \sqrt{a_1^2 + a_2^2 + \dots + a_d^2}$ - $\|\mathbf{x}\|_2^2 = \mathbf{x}^\top \mathbf{x} = \sum_i |x_i|^2$
- $\|\mathbf{x}\|_1 = \sum_i |x_i|$ - $\|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{x}} = \sqrt{\sum_i |x_i|^2}$

• Dot Product: $\langle \mathbf{a}, \mathbf{b} \rangle = \mathbf{a} \cdot \mathbf{b} = \mathbf{a}^\top \mathbf{b} = \|\mathbf{a}\| \times \|\mathbf{b}\| \times \cos(\theta)$

- $\langle \mathbf{x}, M\mathbf{y} \rangle = \langle M^\top \mathbf{x}, \mathbf{y} \rangle$
- $\langle \mathbf{x}, \mathbf{y} + \mathbf{z} \rangle = \langle \mathbf{x}, \mathbf{y} \rangle + \langle \mathbf{x}, \mathbf{z} \rangle$
- Cauchy-Schwarz Inequality: $|\langle \mathbf{u}, \mathbf{v} \rangle| \leq \|\mathbf{u}\| \times \|\mathbf{v}\|$ (Proof by $\min_t F(t) = \|\mathbf{x} + t\mathbf{y}\|^2$, for $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$)
- Random Dot product: Consider $\mathbf{U}, \mathbf{V} \in \mathbb{R}^d$ are i.i.d and $\sim \mathcal{N}(0, \mathbf{I})$
 - * $\mathbb{E}[\langle \mathbf{U}, \mathbf{V} \rangle] = \mathbb{E}[\sum_{i=1}^d U_i V_i] = \sum_{i=1}^d \mathbb{E}[U_i V_i] = 0$
 - * $\mathbb{E}[\langle \mathbf{U}, \mathbf{V} \rangle^2] = \mathbb{E}[\sum_{1 \leq i, j \leq d} U_i V_j] = \sum_{1 \leq i, j \leq d} \mathbb{E}[U_i V_j] = \sum_{1 \leq i, j \leq d} \mathbb{E}[U_i] \mathbb{E}[V_j] = \sum_{i=1}^d \mathbb{E}[U_i]^2 = d$

• Trace: sum of the elements on its diagonal, $\text{Trace}(\mathbf{M}) = \sum_{i=1}^d \mathbf{M}_{i,i}$

- $\text{Trace}(\mathbf{AB}) = \text{Trace}(\mathbf{BA})$
- $\text{Trace}(\mathbf{A}) = \text{Trace}(\mathbf{A}^\top)$
- $\text{Trace}(\mathbf{AB}^\top) = \sum_{i,j} \mathbf{A}_{i,j} \mathbf{B}_{i,j}$
- $\text{Trace}(\mathbf{ABC}) = \text{Trace}(\mathbf{BCA}) = \text{Trace}(\mathbf{CAB}) \neq \text{Trace}(\mathbf{ACB})$

• Determinant: Signed volume generated by the rows (or columns) of that matrix.

- $\det(\mathbf{A}) = \prod_i^d \lambda_i$ - $\det(\mathbf{A}^\top) = \det(\mathbf{A})$
- $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$ - $\det(c\mathbf{A}) = c^n \det(\mathbf{A})$
- $\det(\mathbf{A}^{-1}) = (\det(\mathbf{A}))^{-1}$
- Determinant of a triangular matrix is the product of the diagonal
- Linear map $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$ transforms a volume $\Omega \rightarrow f(\Omega)$ with $\text{Vol}[f(\Omega)] = |\det(f)| \times \text{Vol}[\Omega]$, expanded by a factor $|\det(f)|$

• Invertible:

- $\mathbf{A}$ invertible $\Leftrightarrow \det(\mathbf{A}) \neq 0 \Leftrightarrow$ Eigenvalue $\lambda_i \neq 0 \Leftrightarrow \text{rank}(\mathbf{A}) = n$
- $\Leftrightarrow \mathbf{Ax} = \mathbf{0}$ have only trivial solution ($\mathbf{x} = \mathbf{0}$)
- $\Leftrightarrow \mathbf{Ax} = \mathbf{b}$ have unique solution
- $(\alpha \mathbf{A})^{-1} = \frac{1}{\alpha} \mathbf{A}^{-1}$
- $\mathbf{A}, \mathbf{B}$ Invertible $\Leftrightarrow \mathbf{AB}$ Invertible (By $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$)
- Orthogonal ($\mathbf{AA}^\top = \mathbf{I}$) $\Rightarrow$ invertible
- $\mathbf{A} \succ 0 \Rightarrow$ Invertible
- $\mathbf{A}, \mathbf{B}$ invertible $\Leftrightarrow \mathbf{A} \pm \mathbf{B}$ Invertible
- $\mathbf{A}^{-1}$ have eigenvalue $\frac{1}{\lambda_i}$

• Eigenvalues/vectors: Consider square matrix $M \in \mathbb{R}^{d,d}$. then $\lambda \in \mathbb{R}$ is an eigenvalue of M with an associated eigenvector $\mathbf{v} \neq \mathbf{0}$ if $M\mathbf{v} = \lambda\mathbf{v}$

- The set of eigenvalues of a matrix is called its **spectrum**
- Real square matrices do not necessarily have real eigenvalues
- Eigenvalues are the solutions of the equation: $\det(M - \lambda \mathbf{I}) = 0$
- If $M \in \mathbb{R}^{d,d}$ then $p(\lambda) = \det(M - \lambda \mathbf{I})$ is a polynomial of degree $d \Rightarrow$ at most d distinct eigenvalues
- Eigenvalues of a triangular matrix are the elements on the diagonal
- Let $\mathbf{A} \in \mathbb{R}^{m,n}, \mathbf{B} \in \mathbb{R}^{n,m}$, the eigenvalue $\lambda \neq 0$ of $\mathbf{AB}$ is also eigenvalue of $\mathbf{BA}$, others are 0
- Consider $\mathbf{A} \in \mathbb{R}^{d,d}$ with d eigenvalues.

$$\begin{aligned} \text{Trace}(\mathbf{A}) &= \sum_{i=1}^d \lambda_i & \det(\mathbf{A}) &= \prod_{i=1}^d \lambda_i \\ \text{If } \mathbf{A} &\succeq 0, \frac{\mathbf{Ax}}{\mathbf{x}^\top \mathbf{x}} & \in [\lambda_{\min}, \lambda_{\max}] \end{aligned}$$

• Diagonalizable Matrix: $M \in \mathbb{R}^{d,d}$ is diagonalizable if $\exists$ invertible matrix $P \in \mathbb{R}^{d,d}$ s.t.

$$M = P \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_d \end{bmatrix} P^{-1}$$

for eigenvalues $\lambda_1, \dots, \lambda_d$

$$\text{Trace}(M) = \lambda_1 + \dots + \lambda_d \quad - \quad \det(M) = \lambda_1 \times \dots \times \lambda_d$$

- If the matrix M is diagonalizable then for any $k \geq 1$

$$M^k = P \begin{bmatrix} \lambda_1^k & 0 & \dots & 0 \\ 0 & \lambda_2^k & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_d^k \end{bmatrix} P^{-1}$$

- **Orthogonal:** A matrix $O \in \mathbb{R}^{d,d}$ is orthogonal if it satisfies $OO^\top = \mathbf{I}$
 - $OO^\top = \mathbf{I} \Leftrightarrow O^{-1} = O^\top$
 - Orthogonal matrices preserve norms: $\forall \mathbf{v} \in \mathbb{R}^d$ we have $\|O\mathbf{v}\| = \|\mathbf{v}\|$
 - The rows/columns are orthonormal
 - Example: $\mathbf{R}(\theta)$ is orthogonal for $\forall \theta \in \mathbb{R}$ and $\mathbf{R}(\alpha + \beta) = \mathbf{R}(\beta)\mathbf{R}(\alpha)$

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

- **Singular Value Decomposition:** Any $M \in \mathbb{R}^{m \times n}$ of rank r can be decomposed as

$$M = U \Sigma V^\top$$

- $U \in \mathbb{R}^{m \times m}, V \in \mathbb{R}^{n \times n}$ are orthogonal
- $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{m \times n}$ and singular values satisfy $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$.
- **Applications:** PCA, noise reduction, data compression, low-rank approximation, recommendation systems.

- **QR Decomposition – Gram-Schmidt procedure:** Any $M \in \mathbb{R}^{m \times n}, m \geq n$ can be written as

$$M = QR$$

- $Q \in \mathbb{R}^{m \times n}$ has orthonormal columns
- If $M \in \mathbb{R}^{n \times n}$ is square: $Q \in \mathbb{R}^{n \times n}$ is orthogonal.
- $R \in \mathbb{R}^{n \times n}$ is upper-triangular.

- **Cholesky Decomposition:** Any $S \succeq 0$ can be expressed as $S = LL^\top$ for some **Lower triangular matrix** $L \in \mathbb{R}^{d,d}$. This decomposition is unique when $S \succ 0$

- **Symmetric Matrices:** $M \in \mathbb{R}^{d \times d}$ is symmetric if $M_{ij} = M_{ji}$.
 - Any symmetric $S \in \mathbb{R}^{d \times d}$ is **orthogonally diagonalizable**:

$$S = U \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_d \end{bmatrix} U^\top$$

with U orthogonal.

- $\forall \mathbf{A} \in \mathbb{R}^{d,d}, \frac{\mathbf{A} + \mathbf{A}^\top}{2}$ is symmetric
- **Applications:** PCA, proof of SVD, stationary point analysis.

- **Anti-Symmetric:** $\mathbf{S}^\top = -\mathbf{S}$ for $\mathbf{S} \in \mathbb{R}^{d,d}$

- $S_{ii} = -S_{ii} = 0$
- $\forall \mathbf{A} \in \mathbb{R}^{d,d}, \frac{\mathbf{A} - \mathbf{A}^\top}{2}$ is anti-symmetric. $\mathbf{A} = \frac{\mathbf{A} + \mathbf{A}^\top}{2} + \frac{\mathbf{A} - \mathbf{A}^\top}{2}$
- $\langle \mathbf{x}, S\mathbf{x} \rangle = \langle S^\top \mathbf{x}, \mathbf{x} \rangle = \langle -S\mathbf{x}, \mathbf{x} \rangle = -\langle \mathbf{x}, S\mathbf{x} \rangle = 0$
- If $\dim d = 2n + 1$, S cannot be inverted.

- **Positive Semi-Definite (psd):** A symmetric matrix $S \in \mathbb{R}^{d \times d}$ is psd if $\langle \mathbf{x}, S\mathbf{x} \rangle \geq 0, \forall \mathbf{x} \in \mathbb{R}^d$ denote as $S \succeq 0$

- does NOT imply that all the entries are positive.
- $\Leftrightarrow$ all its eigenvalues are non-negative.
- Let $P \succeq 0$, for $\mathbf{v} \in \mathbb{R}^d$ the vectors $\mathbf{v}$ and $P\mathbf{v}$ roughly point in the same direction $\Rightarrow \text{angle}(\mathbf{v}, P\mathbf{v}) \leq 90$ Degrees.
- Preconditioning matrix to optimize functions by $P \nabla f$
- if $A \succeq 0$, then $M^\top A M \succeq 0$
- $A^\top A \succeq 0$, if A is invertible, we have $A^\top A \succ 0$
- **Example:** Covariance Matrix $C = \mathbb{E}[(X - \mu)(X - \mu)^\top] \succeq 0$

- **Positive Definite (pd):** A symmetric matrix $S \in \mathbb{R}^{d \times d}$ is pd if $\langle \mathbf{x}, S\mathbf{x} \rangle > 0, \forall \mathbf{x} \in \mathbb{R}^d, \mathbf{x} \neq \mathbf{0}$ denote as $S \succ 0$

- $\Leftrightarrow$ all its eigenvalues are strictly positive
- $A \succeq 0, B \succ 0, C = A + B \succ 0$
- $A \succ 0 \Leftrightarrow \exists \mathbf{B}$ invertible, such that $\mathbf{A} = \mathbf{B}^\top \mathbf{B}$
- $S \succ 0 \Rightarrow S^{-1} \succ 0$

Multi-Variable Calculus

- **Gradient:** For $f : \mathbb{R}^d \rightarrow \mathbb{R}$,

$$\nabla f(x) = \begin{bmatrix} \partial_{x_1} f(x) \\ \vdots \\ \partial_{x_d} f(x) \end{bmatrix} \in \mathbb{R}^d.$$

- Gradient gives the direction of steepest ascent.
- Pointing in the direction in which f **increases** the most
- orthogonal to the level lines of f

- **Jacobians:** for $F : \mathbb{R}^p \rightarrow \mathbb{R}^d$

$$F(x + \varepsilon) \approx F(x) + J_F(x)\varepsilon + (\text{error of order } \|\varepsilon\|^2)$$

where $J_F(x) \in \mathbb{R}^{d,p}$ is Jacobian matrix of F at x .

Consider $F : (x_1, x_2, x_3) \rightarrow (y_1, y_2, y_3, y_4) (\mathbb{R}^3 \rightarrow \mathbb{R}^4)$

$$J_F(x_1, x_2, x_3) = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \frac{\partial y_1}{\partial x_3} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \frac{\partial y_2}{\partial x_3} \\ \frac{\partial y_3}{\partial x_1} & \frac{\partial y_3}{\partial x_2} & \frac{\partial y_3}{\partial x_3} \\ \frac{\partial y_4}{\partial x_1} & \frac{\partial y_4}{\partial x_2} & \frac{\partial y_4}{\partial x_3} \end{bmatrix} \in \mathbb{R}^{4,3}$$

- **Hessian:** $d \times d$ matrix of second derivatives

$$\text{Hess}f(x) = \left[\frac{\partial^2 f}{\partial x_i \partial x_j}(x) \right]_{i,j=1}^d$$

- x_0 is a **local minimum** $\Rightarrow \text{Hess}f(x_0) \succeq 0$.
- x_0 is a **saddle point** $\Rightarrow \text{Hess}f(x_0)$ has both positive and negative eigenvalues.
- Give function $f = \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x}$

$$\begin{aligned} \nabla_{\mathbf{x}} f &= \frac{\partial f}{\partial \mathbf{x}} = (\mathbf{A} + \mathbf{A}^T) \mathbf{x} + \mathbf{b} \\ \frac{\partial^2 f}{\partial \mathbf{x} \partial \mathbf{x}^T} &= \mathbf{A} + \mathbf{A}^T \end{aligned}$$

- **First-order approximation:** $f(x + \epsilon) \approx f(x) + \langle \nabla f(x), \epsilon \rangle$.

Example: Reparametrization

Consider $F : \mathbb{R}^d \rightarrow \mathbb{R}$, $M \in \mathbb{R}^d$ and $G(x) = F(Mx)$, we have

$$G(x + \epsilon) \approx G(x) + \langle \nabla G(x), \epsilon \rangle \Leftrightarrow F(M(x + \epsilon)) \approx F(Mx) + \langle \nabla F(Mx), M\epsilon \rangle$$

Therefore we have $G(x + \epsilon) \approx G(x) + \langle M^T \nabla F(Mx), \epsilon \rangle$

Hence, $\nabla G = M^T \nabla F(Mx)$

- **Second-order approximation:**

$$f(x + \epsilon) \approx f(x) + \langle \nabla f(x), \epsilon \rangle + \frac{1}{2} \langle \epsilon, \text{Hess}f(x)\epsilon \rangle$$

- **Taylor Theorem:** Let $F : \mathbb{R}^p \rightarrow \mathbb{R}$ be a smooth function. For $x, y \in \mathbb{R}^p$ there exists $z \in \mathbb{R}^p$ such that

$$F(y) = F(x) + \langle \nabla F(x), (y - x) \rangle + \frac{1}{2} \langle (y - x), H_F(z)(y - x) \rangle$$

- **Linear Function:** For vector $v \in \mathbb{R}^d$ $F_{\text{lin}}(x) = \langle v, x \rangle$

- $\nabla F_{\text{lin}}(x) = v$
- $\text{Hess} = \mathbf{0} \in \mathbb{R}^{d,d}$

- **Quadratic Function:** For a matrix $M \in \mathbb{R}^{d,d}$, $F_{\text{quad}}(x) = \frac{1}{2} \langle x, Mx \rangle$

- $\nabla F_{\text{quad}}(x) = Mx \in \mathbb{R}^d$
- $\text{Hess}_{F_{\text{quad}}} = M^{\text{sym}}$ where $M^{\text{sym}} = \frac{M + M^T}{2}$

- **Gradient Approximation:** Each component of the gradient

$g = \nabla F(x) \in \mathbb{R}^d$ can be approximated as

$$g_i = \lim_{\varepsilon \rightarrow 0} \frac{F(x + \varepsilon e_i) - F(x - \varepsilon e_i)}{2\varepsilon}$$

where $e_i = (0, 0, 0, \dots, 1, \dots, 0)$

- This finite difference approximation costs $2d$ function evaluations

Convexity

- **Convex Function:** A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is

- **Convex** if for any $x, y \in \mathbb{R}^d$ and $0 < \lambda < 1$

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y) \Leftrightarrow \text{Hess}_{F(x)} \succeq 0$$

- **Strictly Convex** if $x \neq y$ and $0 < \lambda < 1$

$$f(\lambda x + (1 - \lambda)y) < \lambda f(x) + (1 - \lambda)f(y) \Leftrightarrow \text{Hess}_{F(x)} \succ 0$$

- * If a strictly convex function has a **local minimum**, this minimum is the **unique global minimum** $\Rightarrow$ Easy to Minimize

- **Strongly Convex:** if F is convex, then $F(x) + \frac{m}{2} \|x\|^2$ is strongly convex, s.t.

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y) - \frac{\mu}{2} \theta(1 - \theta) \|x - y\|^2$$

- Strongly Convex $\Rightarrow$ Strictly Convex $\Rightarrow$ Convex

- **Convexity and Minimal:**

- All local minimal of convex function is global minimal.
- Strictly convex function can have at most one minimal point
- Under convex problem, with valid step, gradient descent/newton will finally converge to a global minimal
- Under non-convex problem(e.g. Rosenbrock), there is high possible that stack in a local minimal or saddle point.

- **Examples of Convex Function:**

- $f(x) = x^2$
- $f(x) = -\log(x)$
- $f(x) = e^x$

- **Operations Preserving Convexity**

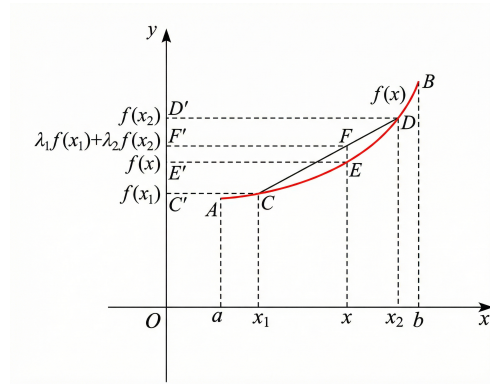
- Sum: $x \mapsto f_1(x) + f_2(x)$ convex.
- Max: $x \mapsto \max(f_1(x), f_2(x))$ convex.
- Composition: if $G : \mathbb{R} \rightarrow \mathbb{R}$ is **convex** and **increasing**, then $G \circ f = G(f(x))$ convex $\Rightarrow f$ can be **non-convex** in this case
- Affine Composite: $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $A \in \mathbb{R}^{n \times m}$, $b \in \mathbb{R}^n$. Define

$$g(x) = f(Ax + b) : \mathbb{R}^m \rightarrow \mathbb{R}$$

- * $f(x)$ convex $\Rightarrow g(x)$ is convex
- * $f(x)$ strongly convex
 - if A invertible $\Rightarrow g$ is strongly convex
 - else g is only convex

- **Jensen's Inequality:** For random variable X and convex F :

$$F(\mathbb{E}[X]) \leq \mathbb{E}[F(X)].$$



- $\mathbb{E}[X]^2 \leq \mathbb{E}[X^2]$.
- $\mathbb{E}[|X|^p]^{1/p} \leq \mathbb{E}[|X|^q]^{1/q}$ for $0 \leq p \leq q$.
- $\mathbb{E}[X] \leq \log \mathbb{E}[e^X]$.
- KL divergence non-negativity:

$$\int p(x) \log \frac{p(x)}{q(x)} dx \geq 0.$$

- **Smooth Function: continuously differentiable** in its domain

- Continuous: for x_0 in its domain we have

$$f(x_0) = \lim_{x \rightarrow x_0^-} f(x) = \lim_{x \rightarrow x_0^+} f(x)$$

- Continuous $\nRightarrow$ Differentiable $\Rightarrow$ Continuous

Naive Gradient Descent

- **Naive Gradient Descent:** Given $f: \mathbb{R}^d \rightarrow \mathbb{R}$, learning rate $\eta > 0$, initial guess x_0

$$x_{n+1} = x_n - \eta \nabla f(x_n)$$

- Can get stuck in local minima.
- Learning Rate Selection
 - * Too small $\Rightarrow$ slow convergence.
 - * Too large $\Rightarrow$ divergence.
- **Classification: Logistic Regression**
 - **Setup**
 - * Dataset: $\{z_i\}_{i=1}^N$ (texts, images, sounds, etc.).
 - * Feature: d -dim vector $x_i = \text{Feature}(z_i) \in \mathcal{X} \equiv \mathbb{R}^p$
 - * Labels: $y_i \in \{1, 2, \dots, C\}$
 - **Goals**
 - * Predict class: $f: \mathcal{X} \rightarrow Y$.
 - * Predict probabilities: $f: \mathcal{X} \rightarrow [0, 1] \times [0, 1] \times \dots \times [0, 1]$.
- **Logistic Regression Model (Binary Classification) Setup**
 - Training dataset with $N \geq 1$ samples $\{x_i, y_i\}_{i=1}^N$:
 - * p -dimensional features: $x_i \in \mathbb{R}^p$
 - * Two classes: $y_i \in \{-1, +1\}$
 - Parameter: vector $\beta \in \mathbb{R}^p$
 - Prediction only depends on the dot product $\langle \beta, x \rangle$

$$\mathbb{P}(y = +1 | x) = \sigma(\langle \beta, x \rangle)$$

$$\mathbb{P}(y = -1 | x) = 1 - \sigma(\langle \beta, x \rangle) = \sigma(-\langle \beta, x \rangle)$$

Sigmoid Function $\sigma(x) = \frac{1}{1+e^{-x}}: \mathbb{R} \rightarrow (0, 1)$. (non-convex)

- * $\sigma(-z) = 1 - \sigma(z)$
- * $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \sigma(z)\sigma(-z)$
- * $\sigma''(z) = \sigma(z)(1 - \sigma(z))(1 - 2\sigma(z))$
- * $g(x) = -\log \sigma(x) \Rightarrow g''(x) = \sigma(x)(1 - \sigma(x)) \geq 0 \Rightarrow$ convex
- **Maximum Likelihood Estimation of $\beta \in \mathbb{R}^p$**
 - Negative log-likelihood (convex and well-behaved):

$$\text{NLL}(\beta) = -\log \left\{ \prod_{i=1}^N \mathbb{P}(y_i | x_i, \beta) \right\} = \sum_{i=1}^N \log(1 + \exp(-y_i \langle \beta, x_i \rangle)).$$
 - *

$$\nabla \text{NLL}(\beta) = \sum_{i=1}^N -y_i x_i \sigma(-y_i \beta^\top x_i) = \sum_{i=1}^N \frac{-y_i x_i}{1 + \exp(y_i \langle \beta, x_i \rangle)}$$
 - Optimal parameter:

$$\beta^* = \arg \min_{\beta \in \mathbb{R}^p} \text{NLL}(\beta).$$

No closed-form solution to β^* since non-linear sigmoid function.

We use gradient descent or newton method to find global minimal.

- **Regularized Logistic Regression**

- Ridge (ℓ_2):

$$\beta^* = \arg \min_{\beta} \text{NLL}(\beta) + \lambda \|\beta\|_2^2$$

$$\|\beta\|_2^2 = \beta_1^2 + \dots + \beta_p^2$$

- Lasso (ℓ_1):

$$\beta^* = \arg \min_{\beta} \text{NLL}(\beta) + \lambda \|\beta\|_1$$

$$\|\beta\|_1 = |\beta_1| + \dots + |\beta_p|$$

- $\lambda > 0$ is a tuning parameter
- Regularization improves generalization and prevents overfitting.

- **Logistic v.s. Linear Regression**

- Ridge Linear Regression

$$\beta_* = \arg \min_{\beta \in \mathbb{R}^p} \|Y - X\beta\|^2 + \lambda \|\beta\|_2^2$$

has closed form: $\beta^* = (X^\top X + \lambda I)^{-1} X^\top Y$.

- Logistic regression requires optimization (**no closed form**).

Gradient Descent Algorithm

- **Gradient Descent Algorithm:** Given $x_0 \in \mathbb{R}^p$, iterate

$$x_{n+1} = x_n + \alpha_n d_n$$

- d_n is a descent direction
- $\alpha_n > 0$ is the step-size (learning rate).
- The choice of α_n may depend on the iteration number n
- **Descent Direction:** Let $F: \mathbb{R}^p \rightarrow \mathbb{R}$ be smooth. A vector $d \in \mathbb{R}^p$ is a **descent direction** at x if

$$F(x + \varepsilon d) < F(x) \quad \text{for all sufficiently small } \varepsilon > 0$$

- Gradient always points in the direction where the function **increase** most $\Rightarrow$ descent direction have **opposite** direction of the gradient

$$\langle d, \nabla F(x) \rangle < 0, \text{ if } \nabla F(x) \neq 0$$
- $d = -\nabla F(x)$ is **steepest descent direction**
 - * Often **not** the best direction
 - * Often almost orthogonal to the direction to the minimum for a “ill-conditioned” function

- **Preconditioner:** For any p.d. matrix $P \in \mathbb{R}^{d,d} \succ 0$,

$$d = -P \nabla F(x)$$

is a **descent direction**.

- Accelerate convergence by $\downarrow$ condition number

Proof

Introduce preconditioner P , new Hessian becomes

$$\tilde{H} = P^{1/2} H P^{1/2}$$

Therefore $\tilde{\lambda}_i$ becomes more centralized,

$$\tilde{\kappa} = \frac{\tilde{\lambda}_{\max}}{\tilde{\lambda}_{\min}} \ll \kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$$

- Ideally $P \approx H^{-1}$ is the best preconditioner
- **Trade-off in Preconditioner**
 - * good and computationally expensive
 - * efficient to compute but less effective
- **Ill-conditioning Problem:** Sensitivity along different directions is very different, leading to numerical instability.
 - Condition Number $\uparrow \Rightarrow$ Harder to optimize
- Occurs when**
 - Data is only informative of a very small subset of the coordinates
 - Highly correlated features (e.g., Linear regression, $X^\top X$ ill-cond.).
 - **High dimensionality** $\Rightarrow$ poorly scaled parameters.
 - Overparameterization (e.g., neural networks) i.e. # of para. $\gg$ the amount of training data $\Rightarrow$ creating flat regions and narrow ridges.

Example: Consider common test **non-convex** Rosenbrock function

$$F(x) = \sum_{i=1}^{d/2} \left[100(x_{2i} - x_{2i-1}^2)^2 + (1 - x_{2i-1})^2 \right]$$

Global minimum at $(1, 1, \dots, 1)$ is ill-conditioned

- **Condition Number:** For a diagonalizable matrix (e.g., Hessian)

$$\kappa(M) = \frac{\lambda_{\max}}{\lambda_{\min}} \geq 1$$

- Large κ indicates **near-singularity** and numerical **difficulty**;
- $\kappa \approx 1$ indicates well-conditioning. (Less Zig-zag)
- The condition number of the **Hessian matrix** at the minimum is a key concept to understand the behavior of the optimization algo.

- **Exact line search:** Natural, but often **impractical**

$$\alpha_n = \arg \min_{\alpha > 0} F(x_n + \alpha d_n)$$

- Large computation cost for high dim
- May stake at local minima for non-convex function
- **Backtracking Line Search:** Pick $\eta \in (0, 1)$ and $\beta \in (0, 1)$. Starting from $\alpha = \alpha_0$, shrink $\alpha \leftarrow \beta \alpha$ until the condition holds:

$$F(x_n + \alpha d_n) \leq F(x_n) + \eta \alpha \langle \nabla F(x_n), d_n \rangle$$
 - Succeeds whenever d_n is a descent direction. (always satisfied)
 - Cannot be used if d is not a descent direction $\Rightarrow$ Infinite loop
- **Convergence Rates:**
 - Clarify applicability (convexity, differentiability, singular minima), qualitative behavior (dependence on initialization, iterate vs. value convergence), and guide practice and experimentation.

- Assume that the function F is to be optimized is **smooth** and **well-behaved**
- Linear rate of convergence means that it is linear on a semi log-scale.

- **Quadratic upper and lower bounds:** Assume F is strongly convex with

$$mI \preceq H_F(x) \preceq MI \quad \forall x \in \mathbb{R}^p$$

We can derive

- Upper Bound: $F(x + \varepsilon) \leq F(x) + \langle \nabla F(x), \varepsilon \rangle + \frac{M}{2} \|\varepsilon\|^2$
- Lower Bound: $F(x + \varepsilon) \geq F(x) + \langle \nabla F(x), \varepsilon \rangle + \frac{m}{2} \|\varepsilon\|^2$

- *Example:* Minimize

$$\varepsilon \mapsto F(x) + \langle \nabla F(x), \varepsilon \rangle + \frac{M}{2} \|\varepsilon\|^2$$

and prove that the gradient descent with $\eta = \frac{1}{M}$ satisfies

$$F(x_{n+1}) \leq F(x_n) - \frac{1}{2M} \|\nabla F(x_n)\|^2 \quad \forall n \geq 0$$

Solution

1. Since by Lojasiewicz-type inequality, $\forall x \in \mathbb{R}^p$

$$F(x^*) \geq F(x) - \frac{1}{2m} \|\nabla F(x)\|^2$$

$\Leftrightarrow$ minimize the quadratic lower bound

2. Consider the steepest gradient descent with $\alpha = \frac{1}{M}$

At each iteration we have

$$F(x_{n+1}) \leq F(x_n) - \frac{1}{2M} \|\nabla F(x_n)\|^2$$

$\Leftrightarrow$ use $\varepsilon = -\alpha \nabla F(x)$ in the quadratic upper bound

$$F(x) + \langle \nabla F(x), \varepsilon \rangle + \frac{M}{2} \|\varepsilon\|^2$$

3. We have

$$\begin{cases} F(x_{n+1}) \leq F(x_n) - \frac{1}{2M} \|\nabla F(x_n)\|^2 \\ F(x_*) \geq F(x_n) - \frac{1}{2m} \|\nabla F(x_n)\|^2 \end{cases}$$

This implies a linear rate of convergence:

$$\log |F(x_n) - F(x_*)| \leq 2 \left(1 - \frac{1}{\kappa}\right)^n \times 2 |F(x_0) - F(x_*)|$$

where $\kappa = M/m$ plays the role of the condition number

- **Convergence Rates: Quadratic Case:** Let $M \geq 0$

$$F(x) = \frac{1}{2} \langle x, Mx \rangle - \langle b, x \rangle$$

Steepest descent with exact line search obeys

$$\|x_n - x^*\|_Q \leq \left(\frac{\kappa(M) - 1}{\kappa(M) + 1} \right)^n \|x_0 - x^*\|_Q, \quad \|x\|_Q^2 = \langle x, Mx \rangle$$

- If $\kappa(M) \gg 1$, the factor $\approx (1 - 1/\kappa)^n \Rightarrow$ very slow convergence
- Inexact line search is usually worse (though not always).

- **Convergence LR:** Consider $F(x) = \frac{1}{2} \langle x, Sx \rangle$, $S \succ 0$ we have

$$\nabla F(x) = Sx, \quad x_{t+1} = x_t - \eta \nabla F(x_t) = x_t - \eta Sx_t$$

Therefore,

$$x_t = (I - \eta S)x_{t-1} = (I - \eta S)^t x_0$$

To ensure convergence to minimum, set $|I - \eta S| < 1$, we have

$$0 < \eta < \frac{2}{\lambda_{\max}}$$

Generally consider a strong convex function F where

$$mI < \text{Hessian} < MI$$

we have convergence constrain:

$$0 < \eta < \frac{2}{M}$$

where $M \Leftrightarrow \lambda_{\max}$

Second Order Methods

- **Quadratic Approximation:** local model, centered around the current estimate of the minimum.
- **Deriving Naive Gradient Descent:** approximation of F in the neighborhood of x_k

$$F(x_k + \varepsilon) \approx F(x_k) + \langle \nabla F(x_k), \varepsilon \rangle + \frac{1}{2\alpha} \|\varepsilon\|^2$$

Equivalent to approximate the Hessian by

$$\text{Hess}_F(x_k) \approx \frac{1}{\alpha} \mathbf{I}_d$$

Minimizing RHS gives the naive gradient descent algorithm

$$x_{k+1} = x_k - \alpha \nabla F(x_k)$$

- **Newton's method:** Minimizing the RHS of second order approx.

$$F(x_k + \varepsilon) \approx F(x_k) + \langle \nabla F(x_k), \varepsilon \rangle + \frac{1}{2} \langle \varepsilon, H_k \varepsilon \rangle$$

gives the basic Newton's algorithm

$$x_{k+1} = x_k - H_k^{-1} \nabla F(x_k)$$

- **Descent Direction:** $d_k = -H_k^{-1} \nabla F(x_k) \Rightarrow H_k^{-1}$ preconditioner
- Hypothesis of **strict convexity** $\Rightarrow H_k \succ 0$, otherwise
 - * H_k cannot be inverted $\Rightarrow$ cannot get newton
 - * Newton may not be descent direction.
- Use **backtracking line search** for choosing α_k

$$x_{k+1} = x_k - \alpha_k H_k^{-1} \nabla F(x_k)$$
 - * $\alpha \rightarrow 1$ near the min (quadratic approx. becomes very accurate)
- Converges **quadratically** once it reaches a neighborhood of min

$$\|x_{k+1} - x^*\| \leq \text{Cst} \times \|x_k - x^*\|^2$$
 - * The number of correct digits roughly **doubles** at each iteration.
 - * Slow at beginning, extremely fast near min.
- **Affine Invariance:** if $\{x_n\}_{n \geq 0}$ is the trajectory of the Newton method when minimizing $F(x)$ started at x_0 , and $\{y_n\}_{n \geq 0}$ is the trajectory of the Newton method when minimizing $G(y)$ started at $y_0 = Ax_0 + b$, then $y_n = Ax_n + b$
 - * The scale and correlation of the coordinates do not matter
 - * Gradient Descent is **not** affine invariant. It is **very sensitive** to the scale of the coordinates
- **Pros:**
 - * Strictly convex function boost convergence (better than GD)
 - * Strongly theoretical guarantee
- **Cons:**
 - * In high-dimension $p \gg 1$, computing the Hessian is **not feasible**
 - * Typically p times more expensive than gradient
 - * Inverse - $O(n^3)$, Storing - $O(n^2)$
 - * Difficult to use sub-sampling techniques to approx. the Hessian
 - * **Strict convexity** is a strong assumption

- **Quasi-Newton Method:** Use the gradient $\nabla F(x_k)$ and the previous approx. B_{k-1} to compute a new approx. B_k of the Hessian

1. Current estimate $\mathbf{x}_k \in \mathbb{R}^p$ and approx. $\mathbf{B}_k^{-1} \in \mathbb{R}^{p \times p}$ of inverse Hess
2. Compute the descent direction $\mathbf{d}_k \in \mathbb{R}^p$:

$$\mathbf{d}_k = -\mathbf{B}_k^{-1} \nabla F(\mathbf{x}_k)$$

3. Use adaptive learning rate strategy to do the update

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$$

4. Update the approximation of the inverse Hessian

$$\left\{ \mathbf{B}_k^{-1}, \nabla F(\mathbf{x}_k), \nabla F(\mathbf{x}_{k+1}) \right\} \rightarrow \mathbf{B}_{k+1}^{-1}$$

- Never compute Hessians. Use all **gradients** computed throughout the algorithm to **approximate** the Hessian or its inverse.
- Constructed $B_k \succ 0$ even if Hessian is not $\Rightarrow$ can be used to minimize **non-convex** functions.
- **Secant Condition:** Let $g_k = \nabla F(x_k)$,

$$B_k(x_k - x_{k-1}) = g_k - g_{k-1}$$
 - * Many ways to choose B_k so that the secant condition is satisfied
 - * Combined with additional constraints (psd, closeness of B_{k+1} to B_k) leads to a variety of quasi-Newton methods.

- **Broyden-Fletcher-Goldfarb-Shanno (BFGS) Update:**

$$B_{k+1} = B_k + \frac{yy^\top}{y^\top s} - \frac{B_k s s^\top B_k}{s^\top B_k s}$$

Inverse approximate Hessian $C_k \equiv B_k^{-1}$

$$C_{k+1} = \left(I - \frac{sy^\top}{y^\top s} \right) C_k \left(I - \frac{ys^\top}{y^\top s} C_k \right) + \frac{ss^\top}{y^\top s}$$

$$* y = g_k - g_{k-1}$$

$$* s = x_k - x_{k-1}$$

* BFGS does not need to invert the Hessian

$$* B_k \succ 0 \Rightarrow B_{k+1} \succ 0$$

- Converge Time:

* Generally **faster** than Newton

* More iterations than newton to converge

* each iteration only costs $O(p^2) < \text{Newton}$

* More space consumption

- **Low-memory BFGS (L-BFGS):**

* Avoids storing the matrices B_k dimension $p \gg 1$.

* Only storing the $L > 1$ past gradients and using them to approx.

- **Gauss-Newton Approximation:** Approximate the Hessian of $F(x) = L(G(x)) : \mathbb{R}^p \rightarrow \mathbb{R}$
 - **Convex function** $L : \mathbb{R}^d \rightarrow \mathbb{R}$, Hessian $\mathbf{H}_L(y)$ easy to compute
 - **Complicated function** $G : \mathbb{R}^p \rightarrow \mathbb{R}^d$, only $J_G(x)$ available
$$\text{Hess}_F(x) = \underbrace{J_G^\top \mathbf{H}_L J_G}_{\text{Gauss-Newton}} + (\text{another term complicated term})$$

Ignore complicated term

$$\text{Hess}_F(x) \approx \hat{\mathbf{H}}_{\text{GN}}(x) = J_G^\top \mathbf{H}_L J_G$$

- $\mathbf{H}_L \succeq 0$ is the Hessian of L at $y = G(x)$
 - * Specifically if $\mathbf{J}$ full rank and $\mathbf{H}_L \succ 0$ we have $\hat{\mathbf{H}}_{\text{GN}} \succ 0$
 - * Otherwise we only have $\hat{\mathbf{H}}_{\text{GN}} \succeq 0 \Rightarrow$ Maybe not a descent direction $\Rightarrow$ add Levenberg-Marquardt regularization
- Useful in **non-linear** regression and **non-linear** classification
- **Levenberg-Marquardt regularization:** for any $D \succ 0$

$$\hat{\mathbf{H}}_{\text{GN}}(x) + \lambda D \succ 0 \Rightarrow \hat{\mathbf{H}}_{\text{GN}} \text{ is invertible} \Leftrightarrow -\hat{\mathbf{H}}_{\text{GN}}(x)^{-1} \nabla F(x) \text{ well define}$$
 - D is often chosen as diagonal
 - * Identity matrix: $D = \mathbf{I}_d$
 - * Diagonal of GN: $D = \varepsilon \mathbf{I}_d + \text{diag}(\hat{\mathbf{H}}_{\text{GN}}(x))$
 - **Descent Direction:** $d = -(\hat{\mathbf{H}}_{\text{GN}}(x) + \lambda D)^{-1} \nabla F(x)$
 - **Choice of λ :** use a backtracking line search to choose λ at each iteration.
 - * Start with an initial guess λ_0
 - * Increase λ until the usual **Armijo condition** is satisfied:
$$F(x + d)^2 \leq F(x) + \rho \langle \nabla F(x), d \rangle, \quad \rho \in (0, 1)$$

• Levenberg-Marquardt Algorithm:

1. Current iterate $\mathbf{x}_k \in \mathbb{R}^p$
2. Compute the **Gauss-Newton approximate Hessian:** $\hat{\mathbf{H}}_{\text{GN}}(\mathbf{x}_k)$
3. Adaptively choose the **regularization parameter** $\lambda_k > 0$
4. Do the update:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - (\hat{\mathbf{H}}_{\text{GN}}(\mathbf{x}_k) + \lambda_k \mathbf{D})^{-1} \nabla F(\mathbf{x}_k)$$

- **Nonlinear Regression:** For complicated function $G : \mathbb{R}^p \rightarrow \mathbb{R}^N$ and a target vector $\mathbf{y} \in \mathbb{R}^N$. Try to minimize

$$F(\mathbf{x}) = \frac{1}{2} \|\mathbf{y} - G(\mathbf{x})\|^2$$

By Gauss-Newton, we have

$$L(\mathbf{z}) = \frac{1}{2} \|\mathbf{z} - \mathbf{y}\|^2, \text{ where } \mathbf{H}_L(\mathbf{z}) = \mathbf{I}_d$$

Gauss-Newton approximation to the Hessian

$$\hat{\mathbf{H}}_{\text{GN}}(\mathbf{x}) = \mathbf{J}_G^\top \mathbf{J}_G$$

- **Rosenbrock Example:** $F(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$ (**non-convex** and **ill-conditioned**) $\Rightarrow$ Gauss-Newton:
 - $G : \mathbb{R}^2 \rightarrow \mathbb{R}^2 : (x_1, x_2) \mapsto (x_2 - x_1^2, 1 - x_1)$
 - $L : \mathbb{R}^2 \rightarrow \mathbb{R} : (y_1, y_2) \mapsto 100y_1^2 + y_2^2 \Rightarrow$ **Convex**

Stochastic Gradient Descent

Consider big data setting $N \gg 1$ with Gradient Descent

$$\nabla F(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N \nabla_{\mathbf{x}} L(\mathbf{x}, y_i)$$

- **Idea:** for **mini-batch size** $n \ll N$, $\{i_1, i_2, \dots, i_n\}$ with random $n \in [1, N]$ consider **stochastic gradient estimate**

$$\widehat{\nabla F}(x) = \frac{1}{n} \sum_{k=1}^n \nabla_{\mathbf{x}} L(\mathbf{x}, y_{i_k})$$

- **Backtracking does not work:** in general, $\widehat{\nabla F}(x_n)$ is **not** a descent direction.
- An epoch corresponds to when **all samples** in the dataset have been seen once.
- Full batch gradient descent corresponds to $n = N$: only **one** parameter update per epoch, which is very slow.
- **Unbiased Estimator:** Since

$$\mathbb{E}[\nabla_{\mathbf{x}} L(\mathbf{x}, y_{i_k})] = \frac{1}{N} \sum_{i=1}^N \nabla_{\mathbf{x}} L(\mathbf{x}, y_i) = \nabla F(\mathbf{x})$$

We have

$$\mathbb{E}[\widehat{\nabla F}(x)] = \frac{1}{n} \sum_{k=1}^n \mathbb{E}[\nabla_{\mathbf{x}} L(\mathbf{x}, y_{i_k})] = \frac{1}{n} \sum_{k=1}^n \nabla F(\mathbf{x}) = \nabla F(\mathbf{x})$$

- **Variance:** Let $G_k := \nabla_{\mathbf{x}} L(\mathbf{x}, y_{i_k})$

$$\begin{aligned} \text{Var}(\widehat{\nabla F}(x)) &= \frac{1}{n^2} \text{Var}\left(\sum_{k=1}^n G_k\right) \\ &= \frac{1}{n^2} \sum_{k=1}^n \text{Var}(G_k) \\ &= \frac{1}{n^2} n \text{Var}(G_1) \\ &= \frac{1}{n} \text{Var}(G_1) = \frac{1}{n} \text{Var}(\nabla_{\mathbf{x}} L(x, y_i)) \end{aligned}$$

- **Trade-offs: choice of n**
 - very good estimate of ∇F but is slow to compute
 - poor estimate of ∇F but is very fast to compute
 - Compute time scales linearly with n to estimate
 - with a GPU, dependence on n is more complicated
- **Fixed Learning Rate:** with a constant step size, the SGD algorithm **does not** converge towards a minimum of F . Typically, the sequence $\{x_n\}_{n \geq 1}$ bounces around a local minimum of F .
- **Learning Rate Annealing:** Start with a high learning rate, and slowly decrease it as the algorithm runs.
- **Cosine Annealing with Warm-Up:** Learning rate is first increased linearly from zero to a maximum value for a few epochs, and then decreased following a cosine schedule. Popular in deep neural networks.
- success depends on tuning hyperparameters like learning rate, batch size, and scheduling

ADAM optimizer: stabilizes and accelerates convergence

- Algorithm:
 1. $m_0 \leftarrow 0, n_0 \leftarrow 0$
 2. $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \widehat{\nabla F}(x_t)$
 3. $\nu_t = \beta_2 \nu_{t-1} + (1 - \beta_2) \widehat{\nabla F}(x_t)^2$
 4. $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$
 5. $\hat{\nu}_t \leftarrow \nu_t / (1 - \beta_2^t)$
 6. $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{\nu}_t} + \epsilon)$
 7. $x_{t+1} = x_t - \alpha \frac{m_t}{\varepsilon + \sqrt{\nu_t}}$
- Hyperparameters:
 - learning rate: $\alpha > 0$
 - exponential decay for gradients: $0 < \beta_1 < 1$
 - exponential decay for squared gradients: $0 < \beta_2 < 1$
- The closer β_1 and β_2 to 1, the more the moving averages are smoothed.
- $\varepsilon > 0$ is a small constant to avoid division by zero.

Momentum: Not a descent algorithm

- **Gradient Descent with momentum:**

$$x_{n+1} = x_n - \alpha \nabla F(x_n) + \beta (x_n - x_{n-1})$$

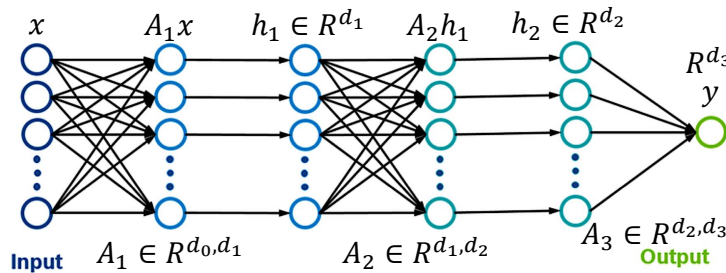
- $-\alpha \nabla F(x_n) + \beta (x_n - x_{n-1})$ is **not** a descent direction
$$\nabla F(\mathbf{x}_n)^\top \mathbf{d}_n = -\alpha \|\nabla F(\mathbf{x}_n)\|^2 + \beta \nabla F(\mathbf{x}_n)^\top (\mathbf{x}_n - \mathbf{x}_{n-1})$$

Since second term may be enough positive
- works for convex and non-convex problem

Deep Learning

Concepts

- **GPU**
 - initially designed for video-games
 - very good at matrix operations and parallel computing
 - often used to accelerate the training process
 - Transfer of data between the CPU and other devices (RAM, GPU) is slow
- **Neural Networks:**
 - Each block represents a differentiable function
 - The overall function is the composition of these blocks
 - Each block is parametrized by a set of weights
 - These weights are learned by **minimizing** a loss function
- Fitting a Deep-Learning model: a **non-convex** optimization problem
- Objective can be **noisy**, **non-differentiable**, and **partly discrete**.
- No guarantee to find the **global** minimum



- # parameter: $d_0 d_1 + d_1 d_2 + d_2 d_3$

Activation Function

- Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$
- Tangent: $\tanh(x)$
- ReLU: $\max(0, x)$
- ELU: $\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$
- Leaky ReLU: $\max(0.1x, x)$

Deep Regression

- **Training Data:** $\{(x_i, y_i)\}_{i=1}^N$
 - scalar response: $y_i \in \mathbb{R}$
 - vector covariate: $x_i = (x_i^{(1)}, \dots, x_i^{(p)}) \in \mathbb{R}^p$
- **Model:** $F_w(x) = w_0 + w_1 x^{(1)} + w_2 x^{(2)} + \dots + w_p x^{(p)} = y$
 - $F_w : \mathbb{R}^p \rightarrow \mathbb{R}$
 - Parameter: $w = (w_0, w_1, \dots, w_p) \in \mathbb{R}^{p+1}$
 - computationally expensive to compute $F_w(x)$
- **Fitting:** w are estimated by minimizing

$$\text{MSE}(w) = \frac{1}{N} \sum_{i=1}^N (y_i - F_w(x_i))^2$$
- Simple algebraic manipulations show that the parameters can be computed in **closed form** with a simple matrix inversion
- **SGD:**
 - use a subset of the data (**mini-batch**) in one iteration
 - **epoch:** all data been processed once
 - A large number of epochs is often necessary for training a neural net
- **Weight Initialization:**
 - Heuristic: random initial values so that the distributions of the neuron activations is roughly centered with unit variance
 - **Lecun init.** Consider a layer with n_{in} input neurons and n_{out} output neurons.

$$w_{ij} \sim \mathcal{N}\left(0, \sigma^2 = \frac{1}{n_{\text{in}}}\right)$$
 - **Xavier init:** $w_{ij} \sim \mathcal{N}\left(0, \sigma^2 = \frac{1}{n_{\text{in}} + n_{\text{out}}}\right)$
 - **He init:** $w_{ij} \sim \mathcal{N}\left(0, \sigma^2 = \frac{2}{n_{\text{in}}}\right)$

Deep Learning Classification

- **Data:** $\{x_i, y_i\}_{i=1}^N$

- Categorical Response: $y_i \in \{0, 1, \dots, C-1\}$
- Input: $x_i \in \mathcal{X}$ (eg. images, documents, time-series, etc)

Prob Classification:

$$x \rightarrow z \in \mathbb{R}^C \rightarrow \sigma(z) = \hat{y} \equiv (p_0(x), p_1(x), \dots, p_{C-1}(x)) \in \mathcal{P}_C \subset \mathbb{R}^C$$

The softmax operation $\sigma : \mathbb{R}^C \mapsto \mathcal{P}_C$ transforms an arbitrary vector into a probability vector

constraints: $\hat{y}_0 + \dots + \hat{y}_{C-1} = 1$

Softmax Operations:

$$\hat{y}_i = \sigma(z) = \frac{\exp(z_i)}{\exp(z_0) + \dots + \exp(z_{C-1})}$$

Log-Sum-Exp: convex function $\mathbb{R}^C \rightarrow \mathbb{R}$

$$\text{LSE}(z) = \log \left\{ \sum_{i=0}^{C-1} \exp(z_i) \right\}$$

– Smooth approximation of the maximum:

$$\max_{0 \leq i \leq C} z_i = \lim_{\varepsilon \rightarrow 0} \varepsilon \times \text{LSE}(z/\varepsilon)$$

Reason: max is not differentiable, we have to approximate it!

– $\sigma(z) = \nabla \text{LSE}(z)$

Fitting: Minimizing NLL (Cross-Entropy Loss)

$$\text{Loss}(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N \log \mathbb{P}(y_i | x_i, \mathbf{w})$$

- $\mathbb{P}(y_i | x_i, \mathbf{w})$: likelihood of data y_i given model parameters $\mathbf{w}$.
- **non-convex** optimization problem
- Typically done using a variation of Stochastic Gradient Descent

From NLL to Cross-Entropy loss:

$$-\log \mathbb{P}(Y = k | \hat{\mathbf{y}}) = -\log(\hat{y}_k) = -\sum_{j=0}^{C-1} y_j^{\text{Hot}} \log(\hat{y}_j) = \text{CE}(\mathbf{y}^{\text{Hot}}, \hat{\mathbf{y}})$$

where one-hot encoding of $y = k$ is a probability vector

$$\mathbf{y}^{\text{Hot}} = (0, 0, \dots, 0, 1, 0, \dots, 0)$$

We have Cross-Entropy between two probability vectors $t, p \in \mathcal{P}^C$

$$\text{CE}(\mathbf{t}, \mathbf{p}) = -\sum_{k=0}^{C-1} t_k \log(p_k) \geq 0.$$

Convolutional Neural Networks

- **max-pooling:**
 - used to **down-sample** the data.
 - allows neural network to abstract the details of the image and understand the **global** structure of the image.
- **global max-pooling:** convert this tensor into a vector of length D
 - applied after processed by a series of convolutions
 - obtained by taking maximum value of each feature map
- **Data Augmentation:** perturbing the training samples.
 - helps mitigate overfitting
 - crucial in situations when training data is limited $\Rightarrow \uparrow$ dataset size
- **Dimension Trick:** Consider input $W_{\text{in}} \times H_{\text{in}} \times D_{\text{in}}$, kernel $K \times K \times D_{\text{kernel}}$, Padding P , Stride S we have output
 - $W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - K + 2P}{S} \right\rfloor + 1$
 - $H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K + 2P}{S} \right\rfloor + 1$
 - $D_{\text{out}} = D_{\text{kernel}}$
 - Trainable Parameter: $(W_{\text{in}} \times H_{\text{in}} \times D_{\text{in}} + 1) \times D_{\text{out}}$

Practical

- If possible, use standard neural architecture that have been proven to work
- Quality and quantity of data is often crucial, more than architecture
- The choice of the optimizer is often not extremely important
- Data-augmentation is often an easy way to improve the performance
- Identify bottlenecks (eg. slow data transfer or data-augmentation)
- Start with simple neural architectures before moving to complex ones
- There are many situations where deep-learning is not the right tool

Bayesian Optimization

Gradient Method	Bayesian Optimization
Exploit local information	Model the function itself
follow the slope	learn the landscape
optimization by iteration	optimization by inference
use many cheap evaluations to estimate the slope	uses few expensive evaluations to update a global model (small budget)
Smooth and Differentiable objectives	each function call takes seconds, minutes, or hours (expensive)
High-dim spaces (BO struggles)	repeated runs give different results.
Simple & Convex : deterministic methods converge faster .	No gradients : the objective is not differentiable or defined by a black box.

Aim: problems too expensive or too opaque for gradients

- Optimization becomes a process of learning where the optimum is **likely** to be
- **Model** summarizes what we know
- **Acquisition** tells us what to try next.

Black-Box Optimization Problem: We can observe inputs and outputs, but not the mechanism inside.

$$\min_{x \in \mathcal{X}} F(x) \quad \text{where} \quad F(x) = f(x) + \varepsilon$$

Goal: Find good points using as **few** evaluations as possible.

- **Expensive**: each evaluation of $F(x)$ is costly.
- **Noisy**: measurements can be corrupted by noise ε .
- **No gradients**: $\nabla F(x)$ is often unavailable.
- **Small budget**: only a few evaluations are allowed.
- No analytic form, no derivatives, no access to internal structure.
- Evaluations are treated as experiments

Function Modeling: build a model of based on the points we have already evaluated. (Foundation of Bayesian Optimization)

- The model gives
 - an estimate of the function value at any point
 - an uncertainty showing where we know little
- Guides **predict**, **interpolate** and **choose** where to explore **next** to best balances two effects
 - **Exploration**: sample where model is uncertain (large σ)
→ potential surprise
 - **Exploitation**: sample where model predicts low values (small μ)
→ likely improvement
- Modeling lets us infer what happens between evaluations.
- Each evaluation is expensive $\Rightarrow$ must learn from few samples.

Model Requirement

- **Flexible**: adapts to complex shapes,
- **Uncertainty-aware**: tells us where we are unsure.
- **Data-efficient**: meaningful predictions from few evaluations.
- **Probabilistic model**: maintain a distribution over possible functions.
- Allows us to reason about both **predictions** and **confidence**.

Gaussian and Uncertainty:

- scalar Gaussian $\rightarrow$ uncertainty over a number
- multivariate Gaussian $\rightarrow$ uncertainty over a vector

Gaussian Process (GP): represents uncertainty over an **entire** function, defined by

- **mean function**: what we expect on average
- **covariance function/kernel**: how points are correlated

$$k(x, x') = \text{Cov}(f(x), f(x'))$$

- Nearby points $\rightarrow$ strong correlation
- Points far apart $\rightarrow$ weak correlation

such that each sample from a GP is a possible function consistent with our prior belief.

- Init: finite set of function values follows a joint Gaussian distribution
- Before seeing data: many possible functions fit our prior belief.
- After observing data: functions must agree with the data.
- Uncertainty shrinks near observations and remains large elsewhere.
- The GP provides

- mean prediction (our best estimate)
- confidence region (how uncertain we are)

Squared Exponential Kernel (RBF Kernel): encodes smoothness, structure, and shape of possible functions

$$k(x, x') = \sigma^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right)$$

Hyper-parameters (typically learned from data)

- Scale parameter σ : controls the "vertical" standard deviation
- Length-scale parameter ℓ : how quickly correlations decay with distance.
 - Small: **wiggly** func. (decorrelation happens quickly)
 $\ell \rightarrow 0, K_\ell \rightarrow \sigma_f^2 \mathbf{I}_n$

Intuition

- * The prior has very **short correlation length** $\Rightarrow$ GP believes that **function changes abruptly** from one point to the next.
- * Each observation affects predictions only in a very small neighborhood
- * The posterior tends to interpolate **perfectly at training points** but fails to generalize smoothly (a behavior similar to overfitting)
- * The acquisition function becomes **highly local** (nearby points are **uncorrelated**, the algorithm **exploits** current best points **instead of exploring** broadly)
- * GP behaves like nearest-neighbor interpolation a extremely flexible but **noisy**.
- Large: **smooth** func. (points stay correlated over longer distance)
 $\ell \rightarrow \infty, K_\ell \rightarrow \sigma_f^2 \mathbf{1}\mathbf{1}^\top$
where $\mathbf{1}\mathbf{1}^\top$ is $n \times n$ matrix of all ones.

Intuition:

- * The prior heavily constrains the function to vary **slowly** assuming no structure in the data.
- * Observations have a **global influence**: learning one value informs all others equal $\Rightarrow$ Aq is flatten. Predictive uncertainty remains roughly constant except where obs $\downarrow$ variance locally.
- * The posterior mean becomes close to the empirical average of observed $\hat{y}_i$, reflecting a **global constant** belief.
- * The acquisition function tends to **favor exploration** initially, since all points are considered equally promising before sufficient data break the near-constant prior.
- * GP model extremely smooth but unresponsive to data and underfitting.

Sampling from GP: Samples reflect the smoothness and structure encoded by the kernel.

1. Choose a set of input points: $X = [\mathbf{x}_1, \dots, \mathbf{x}_n]$.
2. Compute the mean vector $m_i = m(\mathbf{x}_i)$ and the covariance matrix $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$
3. Function values follow a multivariate Gaussian
 $f(X) \sim \mathcal{N}(m, K) \in \mathbb{R}^n$
4. Each sample of $f(X)$ corresponds to **one possible** function drawn from the GP such that

$$f(x_i) \sim \mathcal{N}(m_i, k(x_i, x_i))$$

Acquisition Function: decide where to evaluate next

- sits on top of the probabilistic model and converts GP predictions $(\mu(x), \sigma(x))$ into **utility score** for each candidate point
- encodes the exploration–exploitation trade-off
- high where the next evaluation is most useful
- each evaluation is both an experiment and a decision under uncertainty.
- bridges modeling (inference) and decision (optimization)
- cheap to evaluate and often high multimodal

Expected Improvement: Let the current best observed value be f_{best}
 $\text{EI}(x) = \mathbb{E}[\mathbf{I}(x)]$

- $\mathbf{I}(x) = \max(0, f_{\text{best}} - f(x))$: Improvement obtained by sampling at x . Measure how much lower the objective at x is compared to f_{best}
- EI is large when a point is either exploration and exploitation (naturally balances)
- Models expected gain over current best

- **Closed Form:**

$$\begin{aligned} \text{El}(x) &= (f_{\text{best}} - \mu(x)) \Phi(z) + \sigma(x) \varphi(z) \\ - z &= [f_{\text{best}} - \mu(x)] / \sigma(x) \\ - \varphi(z) &= \text{gaussian PDF at } z = e^{-\frac{1}{2}z^2} / \sqrt{2\pi} \\ - \Phi(z) &= \text{CDF at } z = \int_{-\infty}^z \varphi(t) dt \end{aligned}$$

Lower Confidence Bound (LCB): simple linear combination of mean and uncertainty

$$\text{LCB}(x) = \mu(x) - \kappa \sigma(x)$$

- $\kappa > 0$ controls the exploration–exploitation balance
 - small $\rightarrow$ trust the mean $\rightarrow$ exploit
 - large $\rightarrow$ favor uncertainty $\rightarrow$ explore
 - Typically $\kappa \in [1, 3]$
- Very simple, cheap to compute, and theoretically grounded

Bayesian Optimization Algorithm:

1. **Input normalization:** Rescale each input dimension to $[0, 1]^d$.
(Simplifies acquisition optimization and hyperparameter learning.)
2. **Initial design:** Start with N_{init} random points in $[0, 1]^d$.
 - Usually $N_{\text{init}} \in [5, 20]$.
 - Initialize dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N_{\text{init}}}$.
3. **Bayesian Optimization Loop (Outer loop):** while budget not exhausted do **while True**:
 - (a) **Data standardization (Output normalization):**
Standardize the output values y in $\mathcal{D}$, e.g.

$$\tilde{y} = \frac{\text{Output} - \mu}{\sigma}.$$
 (Stabilizes hyperparameter optimization and prevents numerical issues.)
 - (b) **Model fitting and re-fitting:**
 - i. Update the model using all past evaluations.
 - ii. Fit a GP to the current (standardized) data $\mathcal{D}$.
 - iii. Update GP hyperparameters (e.g., via log-marginal likelihood maximization),
i.e. re-estimate

$$\theta^* = \underset{\theta}{\operatorname{argmax}} \log p(y \mid X, \theta).$$

- Keeps uncertainty estimates calibrated.
 - Prevents overconfidence as new regions are explored.
 - **Notes:** only refit **every few** iterations as it can be **costly**.
- (c) **Acquisition optimization (Inner loop):** cheaply optimize the acquisition $\text{Aq}(x)$ (also written $\alpha(x)$).
 - i. Compute the acquisition function $\text{Aq}(x)$ from the GP.
 - ii. **Multi-start strategy:**
 - A. Draw $N \gg 1$ random initial points in $[0, 1]^d$.
 - B. Run a fast local optimizer from each.
 - C. Keep the best solution found.
 - iii. Select the most promising next point (explore + exploit) by maximizing the acquisition function:

$$x_{\text{next}} = \underset{x \in [0, 1]^d}{\operatorname{argmax}} \text{Aq}(x \mid \mathcal{D}).$$

(Optimization: use multi-start methods to handle the non-convex landscape.)

- (d) **Evaluate and update (Outer loop step):**
 - i. Evaluate the expensive objective at the selected point (possibly noisy):

$$y_{\text{next}} = f(x_{\text{next}}) \quad \leftarrow \text{expensive (+ noisy)}.$$
 - ii. Add new data to the set of evaluations:

$$\mathcal{D} \leftarrow \mathcal{D} \cup \{(x_{\text{next}}, y_{\text{next}})\}.$$

Appendix 1 Vector Calculus

General Case

- $\frac{\partial \mathbf{x}^T \mathbf{a}}{\partial \mathbf{x}} = \frac{\partial \mathbf{a}^T \mathbf{x}}{\partial \mathbf{x}} = \mathbf{a}$
- $\frac{\partial \mathbf{a}^T \mathbf{X} \mathbf{b}}{\partial \mathbf{X}} = \mathbf{a} \mathbf{b}^T$
- $\frac{\partial \mathbf{a}^T \mathbf{X} \mathbf{a}}{\partial \mathbf{X}} = \frac{\partial \mathbf{a}^T \mathbf{X}^T \mathbf{a}}{\partial \mathbf{X}} = \mathbf{a} \mathbf{a}^T$

- $\frac{\partial \mathbf{x}^T \mathbf{B} \mathbf{x}}{\partial \mathbf{x}} = (\mathbf{B} + \mathbf{B}^T) \mathbf{x}$
- $\frac{\partial \mathbf{b}^T \mathbf{X}^T \mathbf{X} \mathbf{c}}{\partial \mathbf{X}} = \mathbf{X}(\mathbf{b} \mathbf{c}^T + \mathbf{c} \mathbf{b}^T)$
- $\frac{\partial (\mathbf{B} \mathbf{x} + \mathbf{d})^T \mathbf{C} (\mathbf{D} \mathbf{x} + \mathbf{d})}{\partial \mathbf{x}} = \mathbf{B}^T \mathbf{C} (\mathbf{D} \mathbf{x} + \mathbf{d}) + \mathbf{D}^T \mathbf{C}^T (\mathbf{B} \mathbf{x} + \mathbf{d})$
- $\frac{\partial}{\partial \mathbf{X}} (\mathbf{X} \mathbf{b} + \mathbf{c})^T \mathbf{D} (\mathbf{X} \mathbf{b} + \mathbf{c}) = (\mathbf{D} + \mathbf{D}^T) (\mathbf{X} \mathbf{b} + \mathbf{c}) \mathbf{b}^T$
- $\frac{\partial \mathbf{b}^T \mathbf{X}^T \mathbf{D} \mathbf{X} \mathbf{c}}{\partial \mathbf{X}} = \mathbf{D}^T \mathbf{X} \mathbf{b} \mathbf{c}^T + \mathbf{D} \mathbf{X} \mathbf{c} \mathbf{b}^T$

Assume $\mathbf{W}$ is symmetric, then

- $\frac{\partial}{\partial \mathbf{s}} (\mathbf{x} - \mathbf{A} \mathbf{s})^T \mathbf{W} (\mathbf{x} - \mathbf{A} \mathbf{s}) = -2 \mathbf{A}^T \mathbf{W} (\mathbf{x} - \mathbf{A} \mathbf{s})$
- $\frac{\partial}{\partial \mathbf{x}} (\mathbf{x} - \mathbf{A} \mathbf{s})^T \mathbf{W} (\mathbf{x} - \mathbf{A} \mathbf{s}) = 2 \mathbf{W} (\mathbf{x} - \mathbf{A} \mathbf{s})$
- $\frac{\partial}{\partial \mathbf{A}} (\mathbf{x} - \mathbf{A} \mathbf{s})^T \mathbf{W} (\mathbf{x} - \mathbf{A} \mathbf{s}) = -2 \mathbf{W} (\mathbf{x} - \mathbf{A} \mathbf{s}) \mathbf{s}^T$

L1 Norm Derivative

Consider

$$f(\mathbf{x}) = \|\mathbf{x}\|_1 = \sum_i |x_i|$$

if and only if when $\forall x_i \neq 0$, we have

$$\nabla \|\mathbf{x}\|_1 = (\text{sign}(x_1), \dots, \text{sign}(x_d))^T = \text{sign}(\mathbf{x})$$

Using chain rule, for $\psi: \mathbb{R}_+ \rightarrow \mathbb{R}$, if and only if when all $x_i \neq 0$

$$\nabla [\psi(\|\mathbf{x}\|_1)] = \psi'(\|\mathbf{x}\|_1) \text{sign}(\mathbf{x})$$

for example

$$\nabla \frac{1}{\|\mathbf{x}\|_1} = -\frac{\text{sign}(\mathbf{x})}{\|\mathbf{x}\|_1^2}$$

$$\nabla \log \|\mathbf{x}\|_1 = \frac{\text{sign}(\mathbf{x})}{\|\mathbf{x}\|_1}$$

$$\nabla \|\mathbf{A} \mathbf{x} - \mathbf{b}\|_1 = \mathbf{A}^T \text{sign}(\mathbf{A} \mathbf{x} - \mathbf{b})$$

L2 Norm Derivative

Consider

$$g(\mathbf{x}) = \|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^T \mathbf{x}}$$

for any $\mathbf{x} \neq \mathbf{0}$, we have

$$\nabla \|\mathbf{x}\|_2 = \frac{\mathbf{x}}{\|\mathbf{x}\|_2}$$

$$\nabla \frac{1}{\|\mathbf{x}\|_2} = -\frac{\mathbf{x}}{\|\mathbf{x}\|_2^3}$$

$$\nabla \log \|\mathbf{x}\|_2 = \frac{\mathbf{x}}{\|\mathbf{x}\|_2^2}$$

$$\nabla \|\mathbf{x}\|_2^2 = 2\mathbf{x} \quad (\text{hold also when } \mathbf{x} = \mathbf{0})$$

Let $h(\mathbf{x}) = \|\mathbf{A} \mathbf{x} - \mathbf{b}\|_2$, for all $\|\mathbf{A} \mathbf{x} - \mathbf{b}\|_2 \neq 0$ we have

$$\nabla g(\mathbf{x}) = \frac{\mathbf{A}^T (\mathbf{A} \mathbf{x} - \mathbf{b})}{\|\mathbf{A} \mathbf{x} - \mathbf{b}\|_2}$$

$$\nabla \frac{1}{\|\mathbf{A} \mathbf{x} - \mathbf{b}\|_2} = -\frac{\mathbf{A}^T (\mathbf{A} \mathbf{x} - \mathbf{b})}{\|\mathbf{A} \mathbf{x} - \mathbf{b}\|_2^3}$$

$$\nabla \log \|\mathbf{A} \mathbf{x} - \mathbf{b}\|_2 = \frac{\mathbf{A}^T (\mathbf{A} \mathbf{x} - \mathbf{b})}{\|\mathbf{A} \mathbf{x} - \mathbf{b}\|_2^2}$$

$$\nabla \|\mathbf{A} \mathbf{x} - \mathbf{b}\|_2^2 = 2 \mathbf{A}^T (\mathbf{A} \mathbf{x} - \mathbf{b}) \quad (\text{hold also when } \|\mathbf{A} \mathbf{x} - \mathbf{b}\|_2 = 0)$$

Appendix 2 Time Complexity

- **Matrix Addition/Transpose/Scale/Dot product:** Consider $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m, n}$

$$T(n) = O(mn)$$

- **Matrix Multiplication:** Consider $\mathbf{A} \in \mathbb{R}^{p, q}, \mathbf{B} \in \mathbb{R}^{q, r}$

$$T(n) = O(pqr)$$

Therefore, the time it takes to multiply matrices depends on the order of operations (brackets)

Specifically, for two square matrix in dim. $n \times n$, we have

$$T(n) = O(n^3)$$

- **Invert Matrix** in dim. $n \times n$

$$T(n) = O(n^3)$$

- **Matrix Decomposition:** Table 1

Decomposition	Form	Condition	Time Complexity
LU	$A = LU$	Square invertible matrix	$O(n^3)$
Cholesky	$A = LL^\top$	Symmetric positive definite matrix	$O(n^3)$
QR	$A = QR$	Any $A \in \mathbb{R}^{m \times n}$	$O(mn^2)$ (if $m \geq n$)
Eigenvalue	$A = V\Lambda V^{-1}$	Diagonalizable square matrix	$O(n^3)$
SVD	$A = U\Sigma V^\top$	Any $A \in \mathbb{R}^{m \times n}$	$O(mn \min(m, n))$

Table 1: Matrix Decomposition Time Complexity